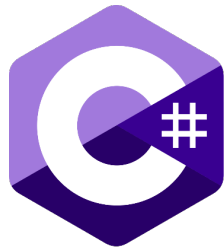
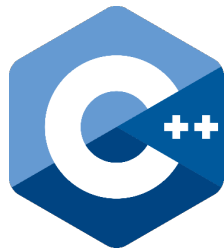


API

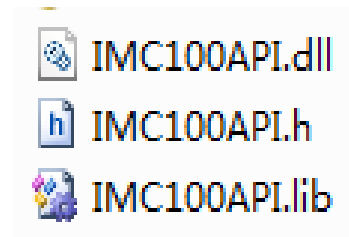
**API**

# **PART 1**

1. API를 통해 다른 PC 프로그래밍 언어를 사용하여 INOVANCE 로봇을 직접 제어할 수 있는 프로그램을 개발할 수 있습니다.
2. 이노벤스 API는 C, C++, C#, VB.NET, LabView 언어들을 지원합니다.
3. 또한 위 지원하지 않는 언어 (Python, Java)에서도 API를 사용할 수 있게 문자열 API 기능(TCP/IP Socket)을 지원합니다.
4. 로봇 컨트롤러의 통신 포트는 항상 2222로 고정됩니다.
5. 더욱 자세한 내용은 API 매뉴얼 (Remote Ethernet Control User Guide)을 참조하세요.



PC



API Function or API String



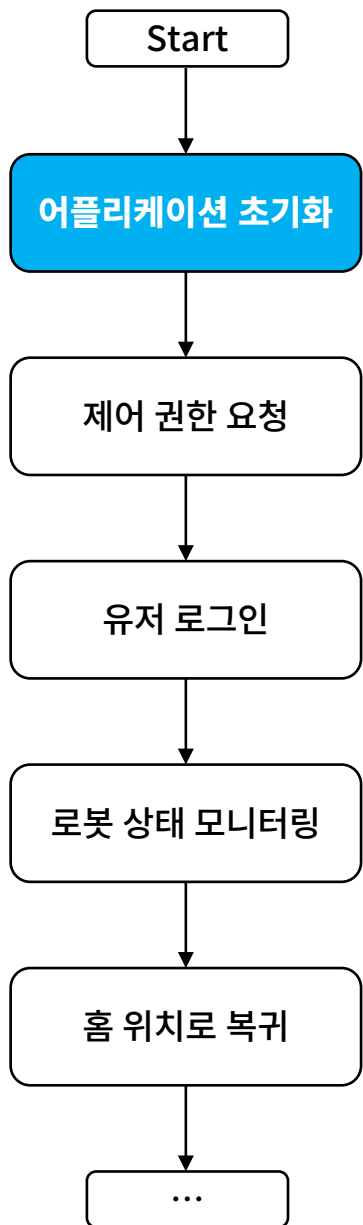
Robot

1. 당사에서 제공하는 IMC100API.lib / IMC100API.dll / IMC100API.h 파일을 프로젝트 경로에 복사합니다.
2. include “IMC100API.h” 구문을 프로그램에 추가합니다.
3. pragma comment(lib, "IMC100API.lib") 구문을 프로그램에 추가하거나 IMC100API.lib 파일을 추가 라이브러리로 추가합니다.
4. API 함수 인터페이스를 호출하여 프로그램을 작성합니다.

## C# 기반 API 설정

1. 당사에서 제공하는 IMC100API.cs, IMC100API.dll 파일을 솔루션 탐색기의 프로젝트 우클릭 -> 추가 -> 기본 항목에서 파일을 프로젝트에 추가합니다.
2. Using IMC100APIDLL; 구문을 프로그램에 추가합니다.
3. API 함수 인터페이스를 호출하여 프로그램을 작성합니다.

1. API는 크게 3가지의 카테고리 (초기화 / 모니터링 / 제어 )로 구분됩니다.
2. 어플리케이션 초기화 : 초기화 함수를 호출합니다.
  - 1) IMC100\_init\_ETH()
  - 2) IMC100\_Exit\_ETH()
3. 모니터링 : 이 유형의 함수는 유저 권한 등의 제한되지 않고 IMC100\_init\_ETH() 함수가 성공적으로 호출된 후 정상적으로 호출할 수 있습니다.
  - 1) IMC100\_Get\_RobPosHere()
  - 2) IMC100\_Get\_DONum()
  - 3) IMC100\_GetStrPara()
  - 4) IMC100\_Get\_RobP() 등
4. 제어 : 이 유형의 함수는 IMC100\_init\_ETH()가 성공적으로 호출된 후에 제어 권한을 Remote EtherNet Client로 전환해야 하며 추가로 IMC100\_AcqPermit() 함수를 호출하여 제어 권한을 얻어야 합니다.
  - 1) IMC100\_MovJ\_P()
  - 2) IMC100\_Set\_DO()
  - 3) IMC100\_Set\_RobP()
  - 4) IMC100\_Set\_HomeJPos()를 포함한 기타 함수들은 더 높은 유저 권한을 요구하기 때문에 IMC100\_UserLogin() 함수를 호출하여 높은 권한을 요청할 수 있습니다.

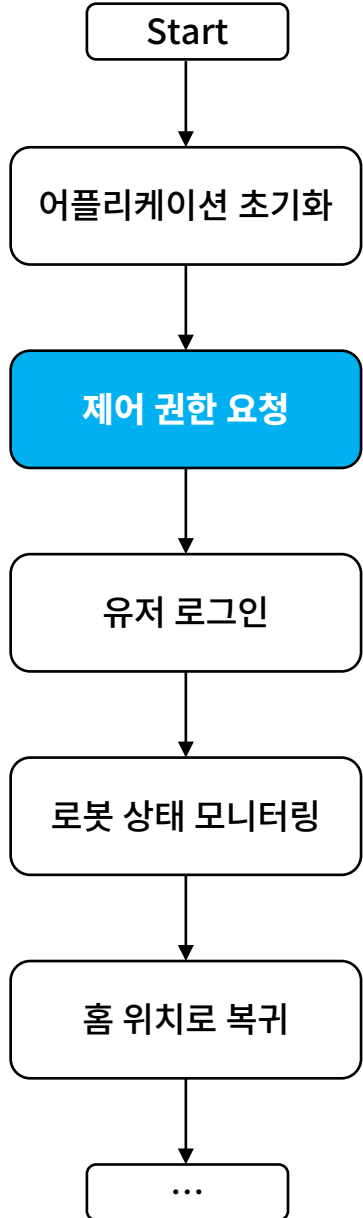


## 1. IMC100\_init\_ETH()

```
int ret = 0;
DWORD dwIP1 = 0xC0 A8 1 19; // Corresponding IP: 192.168.1.25
int ipPort = 2222;
int timeOut = 5; // Communication timeout 5s
int robotNo = 0;
ret = IMC100_Init_ETH(dwIP1, ipPort, timeOut, robot No);
If (ret < 0) {
    // Add the exception handling code here
    Return;
}
```

## 2. IMC100\_Exit\_ETH()

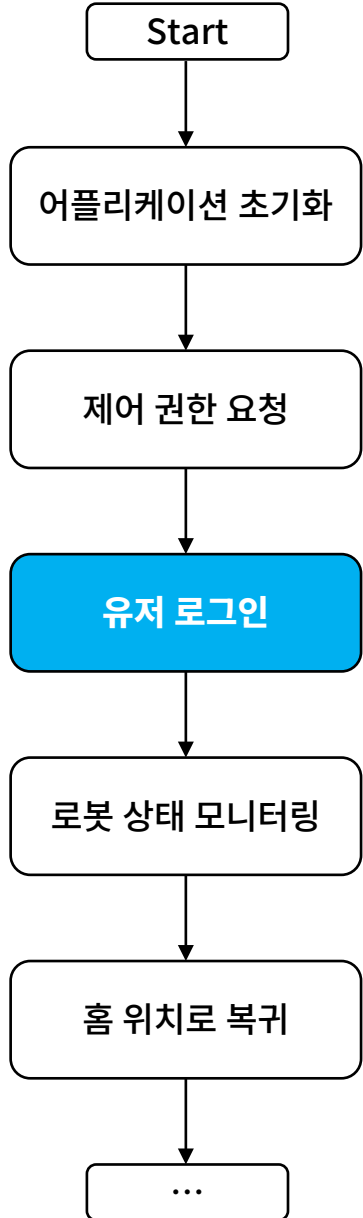
```
ret = IMC100_Exit_ETH(0)
If (ret < 0) {
    // Add the exception handling code here
    Return;
}
```



### 1. IMC100\_AcqPermit() / IMC100\_CurPermit() // 제어 권한 얻기 / 현재 제어 권한 읽기

```
int ret = 0;
int ower = 0;
DWORD ipAddr = 0;
int ipPort = 0;
int robotNo = 0;
ret = IMC100_CurPermit(&ower, &ipAddr, &ipPort, robot No);
If (ret < 0) {
    // Add the exception handling code here
    Return;
}

If (ower != 1) { // The current client device is not licensed
    ret = IMC100_AcqPermit(1, robotNo); // mandatory licensing
    If (ret < 0) {
        // Add the exception handling code here
    }
}
```



### 1. IMC100\_CurUserType() / UserLogin / UserLogout // 현재 유저 권한 조회 / 로그인 / 로그아웃

```
int ret = 0;
```

```
int type = 0;
```

```
char password[8];
```

```
int robotNo = 0;
```

```
ret = IMC100_CurUserType(&Type, robotNo);
```

```
If (ret < 0) {
```

```
    // Add the exception handling code here
```

```
}
```

```
Type = 2;
```

```
memcpy (password, "000000", sizeof(password));
```

```
ret = IMC100_UserLogin(type, password, robotNo); // Login management mode, P/W is same as TP
```

```
If (ret < 0) {
```

```
    // Add the exception handling code here
```

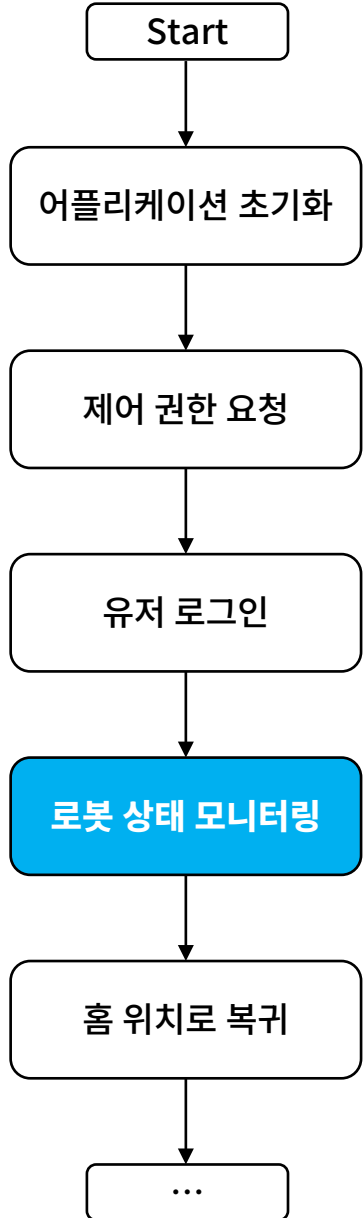
```
}
```

```
ret = IMC100_UserLogout(robotNo);
```

```
If (ret < 0) {
```

```
    // Add the exception handling code here
```

```
}
```



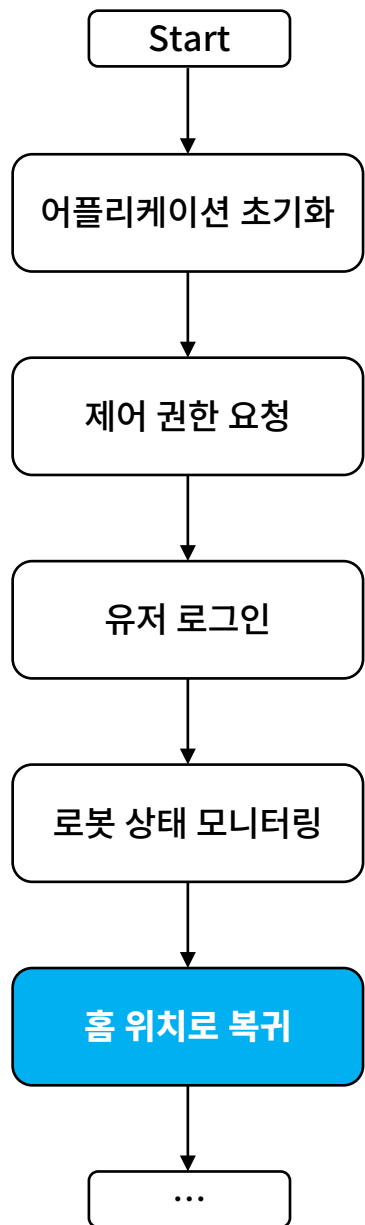
1. IMC100\_Get\_RobPosHere() // Wobj 좌표계 기준으로 현재 위치 값 읽기

```
int ret = 0;
int dinum = 0;
Int dists = 0;
int robotNo = 0;
ROB_POS posTemp;
memcpy (&posTemp, 0, sizeof(posTemp));

ret = IMC100_Get_RobPosHere(&posTemp, robotNo);
If (ret < 0) {
    // Add the exception handling code here
}
```

2. IMC100\_Get\_DI()

```
ret = IMC100_Get_DI(dinum, &dists, robotNo);
If (ret < 0) {
    // Add the exception handling code here
}
```



## 1. IMC100\_DsMode() // Data Stream mode 전환

```
int ret = 0;
int sts = 0;
int robotNo = 0;
ret = IMC100_Get_DsMode(&sts, robotNo);
If (ret < 0) {
    // Add the exception handling code here
}
If (sts = 0) { //The Data Stream mode is turned off
    int cmd = 1;
    ret = IMC100_DsMode(cmd, robotNo); //Enable Data Stream mode
    If (ret < 0) {
        // Add the exception handling code here
    }
}
```

## 2. IMC100\_Home()

```
int num = 0;
ret = IMC100_Home(num, robotNo); //Home[0] 위치로 이동
If (ret < 0) {
    // Add the exception handling code here
}
```

## 1. 로봇 속도 변경 예제

```
// 속도 설정
try
{
    int ret = IMC100API.IMC100_Set_Vel(tbar_speed.Value, 0);
    if (ret < 0)
    {
        AppendLog("로봇 속도 설정 실패!");
    }
}
catch (Exception ex)
{
    AppendLog("로봇 속도 설정 시 예외 발생! 예외 메시지: " + ex.Message);
    MessageBox.Show("예외 발생: " + ex.Message, "에러", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

# 따라하기 (C#)

## PART 2

# 1. 프로젝트 생성

1. 예제는 비주얼 스튜디오 2022을 사용하였습니다.
2. 프로젝트를 생성합니다. (Win Forms이 아니어도 무방합니다.)
3. 프레임 워크는 사용자가 선택하여 사용할 수 있습니다.

## 새 프로젝트 구성

Windows Forms 앱 C# Windows 데스크톱

프로젝트 이름(I)  
Test1

위치(L)  
C:\Users\hois5\OneDrive\Desktop\made\_C#

솔루션(S)  
새 솔루션 만들기

솔루션 이름(M) ⓘ  
Test1

☐ 솔루션 및 프로젝트를 같은 디렉터리에 배치(D)

"C:\Users\hois5\OneDrive\Desktop\made\_C#\Test1\Test1"에 프로젝트이(가) 만들어집니다.

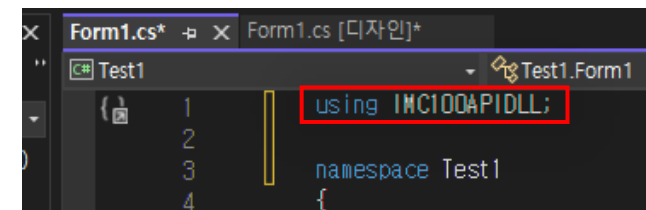
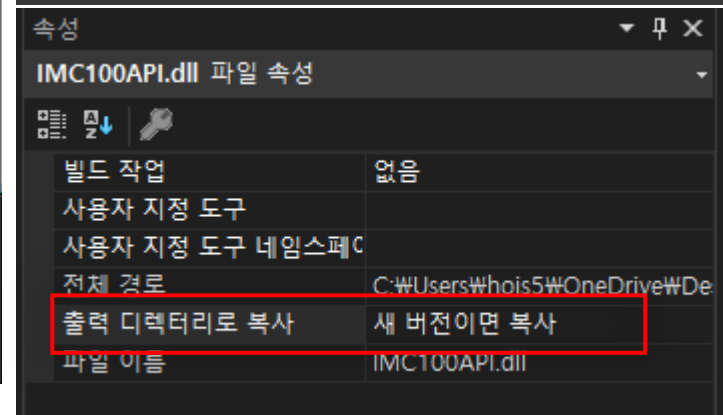
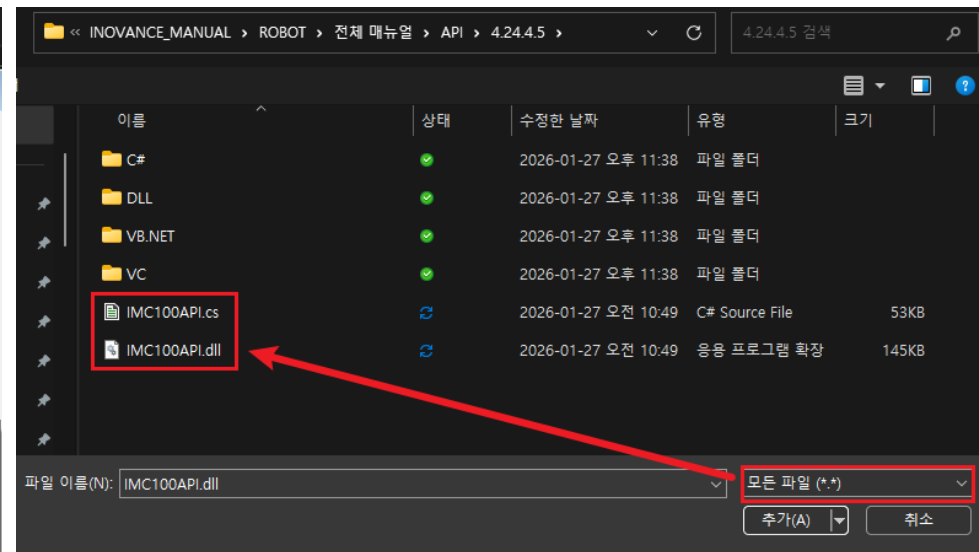
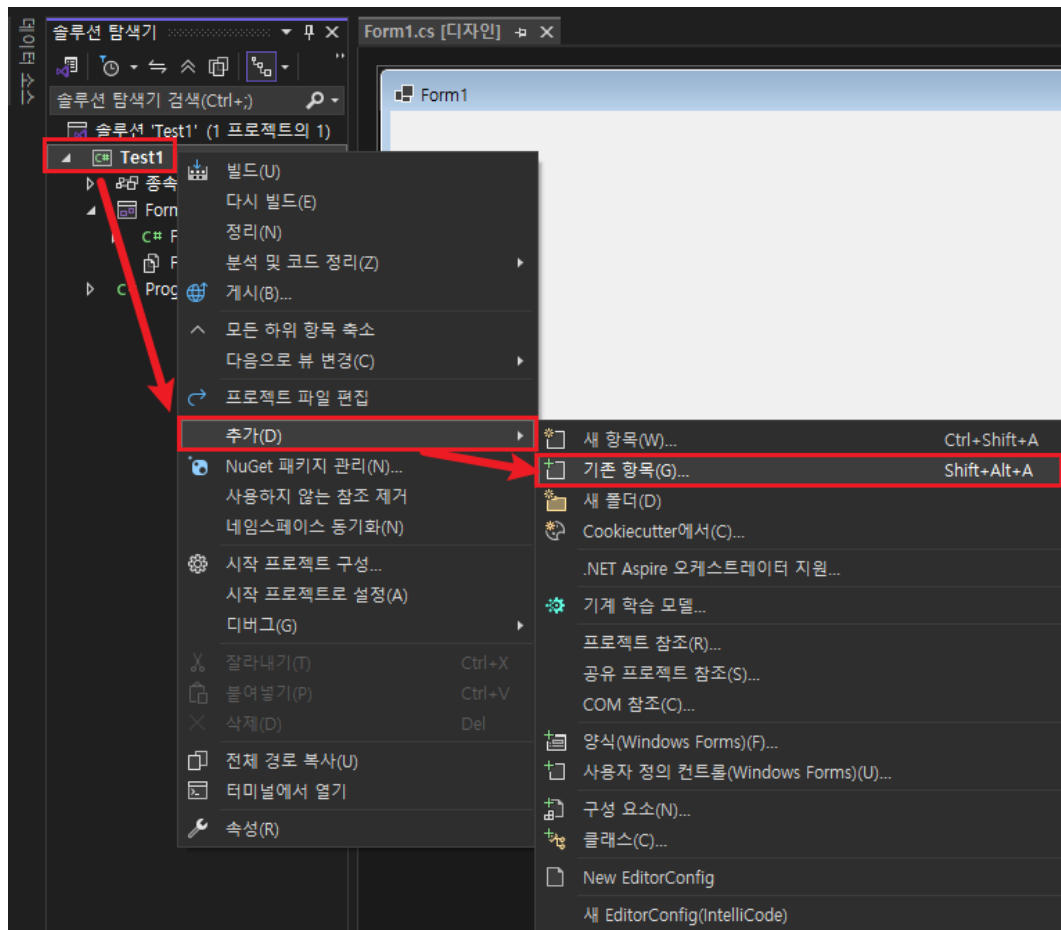
## 추가 정보

Windows Forms 앱 C# Windows 데스크톱

프레임워크(F) ⓘ  
.NET 8.0 (장기 지원)

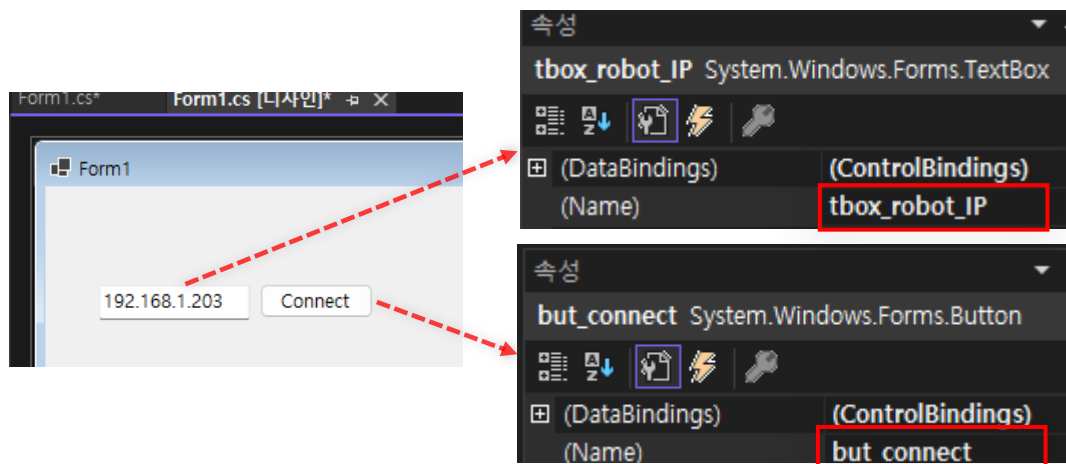
## 2. 라이브러리 추가

1. 프로젝트 우클릭 -> 추가 -> 기존 항목 -> 모든 파일(\*.\*) 선택 -> .cs, .dll 파일들을 추가합니다.
2. dll 파일은 32, 64bit에 맞는 dll 파일을 추가합니다.
3. IMC100API.dll 파일을 선택하고 속성에서 '출력 디렉터리로 복사' 항목을 '새 버전이면 복사'로 변경합니다.
4. 프로그램 상단에 using IMC100APIDLL; 을 작성합니다.



### 3. 로봇과 연결

1. Form1 에 아래와 같이 텍스트 박스와 버튼을 추가하여 사용자에게 로봇 IP를 입력 받고, Connect 버튼을 눌러 로봇과 통신을 시도합니다.



2. Connect 버튼을 두번 클릭하여 클릭 이벤트를 생성한 뒤, 해당 코드를 입력합니다.
3. 로봇 Port는 2222로 고정되어 있으며, Robot\_no도 0으로 설정합니다.
4. tbox\_robot\_IP 박스에 IP를 . 을 포함하여 입력하기 때문에 . 텍스트를 파싱합니다.
5. IMC100API.IMC100\_Init\_ETH 함수를 호출하여 로봇과 연결을 시도합니다.
6. 리턴 값이 0보다 작으면 에러, 0이면 성공입니다. (에러 코드는 매뉴얼을 참조하세요.)

```
참조 1개
private void but_connect_Click(object sender, EventArgs e)
{
    int ret = 0, timeout = 2, robot_no = 0;
    ushort port = 2222;
    string strRobot_IP = tbox_robot_IP.Text; // "192.168.1.203"
    uint iRobot_IP = 0;

    // 1. '.'으로 쪼개서 각각 숫자로 바꾼 뒤 바로 비트 시프트로 합치기
    string[] parts = strRobot_IP.Split('.');

    if (parts.Length == 4)
    {
        iRobot_IP = (uint.Parse(parts[0]) << 24) |
                    (uint.Parse(parts[1]) << 16) |
                    (uint.Parse(parts[2]) << 8) |
                    uint.Parse(parts[3]);

        // 2. 로봇 통신 연결 시도
        ret = IMC100API.IMC100_Init_ETH(iRobot_IP, port, timeout, robot_no);

        if (ret < 0)
        {
            but_connect.Text = "Failed!";
            return;
        }
        else
        {
            but_connect.Text = "Connected!";
        }
    }
}
```

## 4. 제어권한 얻기

1. Form1 에 제어권한 얻기 버튼을 추가하여, 버튼 클릭 이벤트에 제어권한 요청을 시도합니다.
2. Get permit 버튼을 두번 클릭하여 클릭 이벤트를 생성한 뒤, 해당 코드를 입력합니다.

```
private void but_get_permit_Click(object sender, EventArgs e)
{
    // 로봇 권한 얻기

    int ret = IMC100API.IMC100_AcqPermit(1, 0);
    if (ret < 0)
    {
        but_get_permit.Text = "요청 실패!";
        return;
    }
    else
    {
        but_get_permit.Text = "요청 성공!";
    }
}
```

1. Form1 에 로그인 버튼을 추가하여, 버튼 클릭 이벤트에 로그인 요청을 시도합니다.
2. Login 버튼을 두번 클릭하여 클릭 이벤트를 생성한 뒤, 해당 코드를 입력합니다.

```
private void but_login_Click(object sender, EventArgs e)
{
    // 비밀번호 설정
    byte[] password = new byte[6] { (byte)'0', (byte)'0', (byte)'0', (byte)'0', (byte)'0', (byte)'0' };

    // 2: Admin 권한으로 로그인, 패스워드 000000 기본 값
    int ret = IMC100API.IMC100_UserLogin(2, password, 0);

    if (ret < 0)
    {
        but_login.Text = "로그인 실패";
    }
    else
    {
        but_login.Text = "로그인 성공";
    }
}
```

## 6. 에러 리셋, 서보 ON

1. Form1 에 Error reset, Servo ON 버튼을 추가하여, 버튼 클릭 이벤트에 아래 코드를 입력합니다.

참조 1개

```
private void but_error_reset_Click(object sender, EventArgs e)
{
    IMC100API.IMC100_ResetErr(0);
}
```

참조 1개

```
private void but_servo_on_Click(object sender, EventArgs e)
{
    int get_motor_sts = 0;
    int ret = IMC100API.IMC100_Get_MotorSts(ref get_motor_sts, 0); // Servo ON

    if (get_motor_sts == 0)
    {
        ret = IMC100API.IMC100_MotorEnable(1, 0); // Servo ON
        but_servo_on.Text = "Servo ON";
    }
    else
    {
        ret = IMC100API.IMC100_MotorEnable(0, 0); // Servo OFF
        but_servo_on.Text = "Servo OFF";
    }
}
```

## 7. DS mode 전환

1. DS mode는 PC에서 로봇을 직접 제어하기 위해 사용되는 모드로, 로봇을 이동시키기 위해서는 DS mode로 전환되어야 합니다.
2. DS mode로 전환하기 위한 조건은 수동 모드여야 하며, Servo ON상태, 에러 없음 조건을 충족해야 합니다.
3. Form1 에 DS mode 버튼을 추가하여, 버튼 클릭 이벤트에 아래 코드를 입력합니다.

```
private void but_DSmode_Click(object sender, EventArgs e)
{
    int ret = 0;
    int sts = 0; // 0: OFF, 1: ON
    int robotNo = 0;

    ret = IMC100API.IMC100_Get_DsMode(ref sts, robotNo);

    if (sts == 0)
    {
        ret = IMC100API.IMC100_DsMode(1, robotNo);
        if (ret >= 0)
        {
            but_DSmode.Text = "DS Mode ON";
        }
    }
    else
    {
        ret = IMC100API.IMC100_DsMode(0, robotNo);
        if (ret >= 0)
        {
            but_DSmode.Text = "DS Mode OFF";
        }
    }
    return;
}
```

## 8. 로봇 특정 위치로 이동 전환

1. Form1 에 이동 버튼을 추가하여, 버튼 클릭 이벤트에 아래 코드를 입력합니다.

```
private void but_movj_Click(object sender, EventArgs e)
{
    ROB_POS pos = new ROB_POS();
    VER_DATA vel = new VER_DATA();
    MOV_IO movlo = new MOV_IO();
    int ioNum = 0, comId = 0;
    pos.RPosData = new double[6];
    pos.ArmParm = new int[4];
    pos.ArmParm[3] = 1; // 로봇 자세 설정

    // 위치값 설정
    pos.RPosData[0] = 561; // mm
    pos.RPosData[1] = 0; // mm
    pos.RPosData[2] = 676; // mm
    pos.RPosData[3] = 0; // degree
    pos.RPosData[4] = 0; // degree
    pos.RPosData[5] = -180; // degree

    // 속도값 설정
    vel.velPercent = 100; // 속도 백분율 (0~100)

    // Zone 설정
    int zone = 0;

    IMC100API.IMC100_MovJ_RobPos(ref pos, ref vel, zone, ioNum, ref movlo, comId);
}
```

1. Form1 화면은 아래와 같이 구성되었습니다.

2. 동작 순서 :

- 1) 로봇 IP 입력 후 Connect 누름 -> 해당 버튼 Connected! 확인
- 2) Get permit 누름 -> 해당 버튼 요청 성공! 표시 확인
- 3) Login 누름 -> 해당 버튼 로그인 성공 확인
- 4) Error 발생 시 리셋 누름
- 5) Servo ON 누름 -> Servo ON 확인
- 6) DS mode 누름 -> DS mode ON 확인
- 7) Movj , Movj2 버튼을 눌러 로봇 동작 확인

The screenshot shows a window titled "Form1". It contains a text input field with the value "192.168.1.203". To the right of the input field is a "Connect" button. Below the input field, there are four buttons arranged in two columns: "Error reset" and "Servo ON" on the left, and "Get permit" and "DS mode" on the right. At the bottom right of the window, there are two buttons labeled "Movj" and "Movj2".

The screenshot shows the same "Form1" window after several operations. The "Connect" button is now labeled "Connected!". The "Get permit" button is now labeled "요청 성공!". The "Login" button is now labeled "로그인 성공". The "DS mode" button is now labeled "DS Mode ON" and is highlighted with a blue border. The "Error reset" and "Servo ON" buttons remain unchanged. The "Movj" and "Movj2" buttons are still at the bottom right.

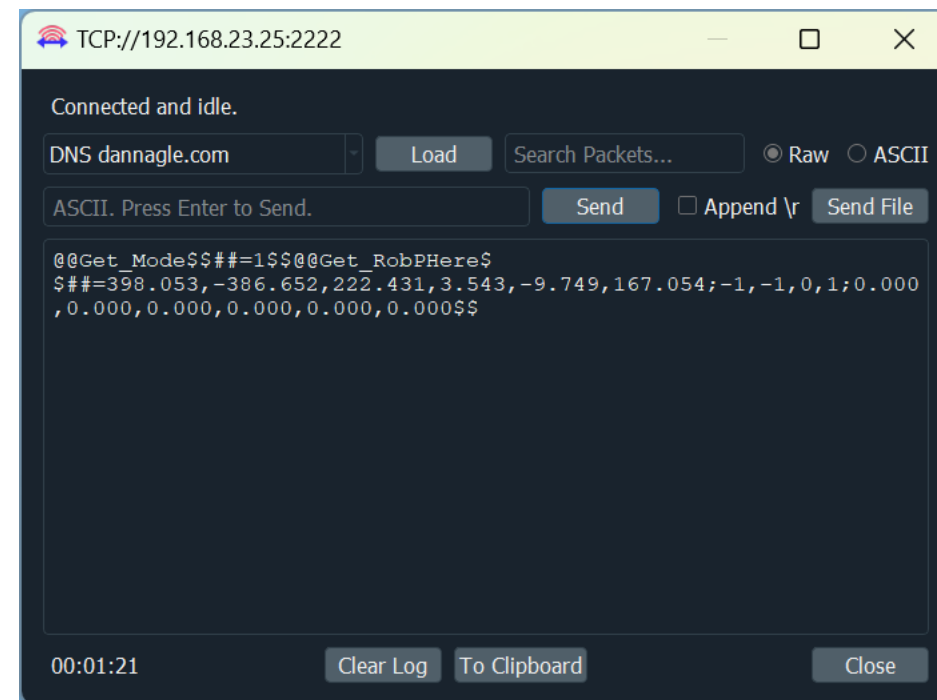
# 문자열 API

## PART 3

1. 문자열 API 기능은 기본 지원 되지 않는 프로그래밍 언어에서 문자열을 통해 API 기능을 사용할 수 있습니다. (TCP/IP Socket)
2. 데이터 송신 형식 : @@ 송신 문자열 \$\$
3. 데이터 수신 형식 : ## 수신 문자열 \$\$
4. 로봇 컨트롤러의 포트는 항상 2222로 고정됩니다. (Server)

## 5. 예제 : Get\_RobPHere (로봇 현재 위치 읽기)

- 1) 송신 문자열 : @@Get\_RobPHere\$\$
- 2) 수신 문자열 : ##=X 위치, Y 위치, Z 위치, A 위치, B 위치, C 위치; 암 파라미터; 외부 축 위치 값\$\$



# INOVANCE

Forward, Always Progressing